



Department of Computer Engineering & Technology
CSE-AIDS — LEVEL 1
Capstone Project Synopsis 2026-27

Group No:	Student names	PRN	Email, Mobile number	Panel	Sign of student
	Raj Modi	1032232965	Raj.modi@mitwpu.edu.in 8817130400	B	
	Krishna Renuse	1032232790	Krishna.renuse@mitwpu.edu.in 8766672821	B	
	Aaditya Kumar Sinha	1032232955	Aaditya.sinha@mitwpu.edu.in 7004389582	B	
	Levinesh G R	1032232546	Levinesh.gr@mitwpu.edu.in 9373334395	B	

Project Title: SynthProof: Synthetic Data That Ships With Its Proof

Project Domain: Differential Privacy and Trustworthy Machine Learning · Generative Models for Tabular Data · Data Governance and Compliance Engineering

Name of the Company (for Industry Supported Project only): N/A. This is an independent in house research project

Name of the External Guide (for Industry Supported Project only): N/A

INHOUSE PROJECT: YES

Abstract

Hospitals, banks and companies cannot share real data because it identifies real people, so they replace it with fake data that looks real. The problem is that nobody checks whether the fake data actually hides anyone; they publish a similarity score and call it a guarantee. SynthProof generates the fake data and then attacks it, so every dataset ships with a receipt showing exactly how hard we tried to break it and failed.

That blocker stops almost every serious data project before it starts. A bank wants a fraud model but is not allowed to export transaction histories. A hospital wants to predict readmissions and cannot release discharge records. Teams get hired to study churn and then spend six weeks in legal review before they see a single row. The old workaround was anonymisation: drop the names, round off the postcodes, ship the file. That has been broken in public many times over, because whatever attributes are left behind are usually rare enough to point at one person. Synthetic data is the modern answer and it is a good one. What has been missing is any way to check the claim.

We are building SynthProof around a single idea. A privacy claim that has never been attacked is not worth very much. You give the platform a sensitive table, or a set of related tables, and it returns a synthetic replacement together with two numbers that nobody currently reports side by side. The first is the theoretical privacy loss. A formal accountant proves it and tracks every operation starting from the moment the schema is read, including the step where column ranges get inferred. Most pipelines do that on the raw data for free and never pay for it. The second number is what an attacker actually managed. We plant random canary records

inside the training run and measure how confidently they can be picked out afterwards. Audits like this used to need hundreds of trained models. A result from 2023 brought the cost down to a single training run, which is why we can afford to do it on every release rather than once in a paper. That same result is worth reading closely. A mechanism proved to satisfy $\epsilon \leq 4$ was measured leaking at least $\epsilon \geq 1.8$. Nobody has checked how wide that gap gets for tabular data, and tabular data is where the commercial privacy claims are being sold.

Three things sit around that core. Privacy budget stops being a flag you pass on the command line and becomes a resource with an account behind it. Every debit is written to a signed ledger covering every release the organisation has ever made, not just the one in front of you. An allocator then decides where the remainder goes, weighting columns and marginals by how much they genuinely contribute to the models the client cares about, because spreading budget evenly across fields nobody will ever use is waste. Generation is plural on purpose. There is a marginal based synthesiser descended from the winning entries of the NIST privacy challenges, a tabular diffusion model trained under differentially private gradient descent, and a fast copula baseline. All three draw on the same accountant, so comparing them is honest. Then come the attacks. Four of them run against every release alongside a control group we held back: shadow model membership inference, attribute inference by linear reconstruction, the singling out and linkability and inference tests that regulators actually reference, and a density ratio membership attack. Finally the frontier engine sweeps the privacy parameter and draws the resulting curve. Every point on it carries the audited number and a set of utility measures rather than one score: accuracy of a model trained on synthetic data and tested on real data, marginal distances, correlation error, and how far the fairness gaps moved. A dataset that holds its overall accuracy while flattening a minority group has not kept its utility.

What we ship is a web application. An analyst uploads a sensitive extract, picks a point on the curve, and downloads three things. The synthetic dataset. A Privacy Data Sheet, machine readable, stating the mechanism used, the budget spent and the audited bound. And the full attack dossier, listing every adversary we ran and exactly how it performed. The demonstration is meant to be slightly confrontational. We take an attack that succeeds against a naively generated dataset, change nothing about it, point it at a SynthProof release, and watch it fail. That failure goes into the certificate rather than onto a slide. So there are two contributions here. One is engineering. Generation, formal accounting, empirical auditing and adversarial testing arrive as one artefact instead of four separate libraries that a practitioner is expected to wire together correctly. The other is a measurement. How far apart do the proved and the achieved privacy loss actually sit, across different generator families and differently shaped datasets? The auditing literature has answered that for image classifiers. For the tabular case, which is what industry ships, it is still open.

Novelty and Research Contribution

- **Privacy accounted from both sides.** The tools that generate private tabular data and the code that audits privacy currently live in different repositories. SmartNoise generates under a formal accountant but ships no auditor. SDV, the library most people reach for first, has no private path at all. Synthcity offers the widest catalogue of generators and hands privacy measurement off to libraries such as Anonymeter, which attack the output while knowing nothing about the mechanism that produced it. We put both halves in one pipeline. Every release carries a proved bound from the accountant and a measured bound from canaries planted in the training loop, and we report the distance between them. The current default is to publish a proof the reader has no way of checking.
- **Budget kept in a ledger.** Everywhere else, ϵ is a per run flag, which quietly assumes this release is the only one you will ever make. It never is. A table synthesised four times at ϵ of 1 is not protected at ϵ of 1, because privacy loss accumulates across releases and nothing in the current toolchain keeps count. We treat budget as a finite resource with an append only signed ledger spanning the whole release programme. The allocator borrows an idea from the strongest marginal mechanism, which adapts its choices inside a single run, and stretches that idea across runs. The ledger ends up doing two jobs at once: it schedules the work, and it is the compliance document.
- **A utility claim you can attack.** Synthetic data quality usually gets published as one similarity number, and recent work has shown similarity based privacy metrics to be actively misleading. Datasets marketed

as genuinely anonymous have fallen to fairly simple attacks. We report utility as a vector along the swept frontier and mark every point with how well four independent adversaries did against a shared control group, so that chance level performance is visible instead of assumed. A number an attack can prove wrong is worth more than a score that cannot.

Minimum Viable Product (MVP)

A single tenant web application. It takes a CSV or a SQL extract, works out a private domain, and generates a synthetic replacement using two interchangeable engines, one marginal based and one a diffusion model trained under private gradient descent. Without being asked, it then runs the auditor, the four attacks and the utility evaluator against what it just produced. The user sees the privacy and utility curve, a live budget ledger and the attack dossier, and downloads the dataset with a signed Privacy Data Sheet. We are judging the MVP on numbers rather than on a demo. At ϵ of 1 on ACSIncome, macro F1 from training on synthetic data and testing on real data has to land within five points of training on real data. All four attacks have to sit at or below 55 percent accuracy against a held out control, where 50 percent is chance. The audited ϵ has to come out automatically on every run, not as a separate experiment. And re running any release from its ledger entry and seed has to reproduce the same data and the same accounting exactly. Without that last one, none of the other three numbers means anything. Multi table synthesis under one global budget, with foreign keys kept intact, is the first thing we add after this.

H/w & S/w Requirements

Frontend	React 18 with Vite, Tailwind CSS, and D3.js or visx for the privacy and utility curve, the ledger timeline and the attack dossier views
Backend and API	Python 3.11, FastAPI, Celery with Redis for the long synthesis and audit jobs, WebSocket streaming for progress
Privacy Core	Opacus for private gradient descent, Rényi and privacy loss distribution accountants (dp-accounting, autodp), Private-PGM for the marginal mechanisms, SmartNoise-Synth
Generative Models	PyTorch 2.x. A latent variational autoencoder with a denoising diffusion head for tabular data, plus Gaussian copula and CTGAN baselines
Audit and Attack Suite	Our own canary auditor, a shadow model likelihood ratio attack, Anonymeter, DOMIAS, and scikit-learn with XGBoost for the downstream evaluation
Baselines and Reporting	Non private CTGAN and Gaussian copula as the utility ceiling, independent marginal sampling as the floor, SDMetrics for fidelity reporting so our results stay comparable with published work
Data and Storage	PostgreSQL for the ledger and run metadata, DuckDB with Parquet for columnar evaluation, MinIO or S3 for release artefacts
Datasets	UCI Adult, the field's default benchmark. ACS PUMS through folktables (Income, Employment, Public Coverage), the modern large scale replacement for Adult. Give Me Some Credit and Bank Marketing for financial class imbalance. MIMIC-IV demo for a clinical schema. Covertypes for high cardinality categoricals
Deployment	Docker Compose across API, worker and database services. Frontend on Vercel, backend on a Linux virtual machine
Hardware	A development laptop with 16 GB RAM. One NVIDIA GPU with 8 to 16 GB of VRAM is recommended for training the diffusion model privately, though the marginal engine runs fine on CPU alone
Testing and PM	pytest with Hypothesis property tests, fixed seed fixtures so the ϵ accounting reproduces, GitHub Actions for CI, Git and GitHub, Notion for sprint tracking

High Level Design [Architecture Diagram / Block Diagram]

The pipeline reads the extract and immediately spends a small, recorded slice of the budget on learning the schema and the column ranges, because working those out from raw data for free is a leak that most pipelines commit silently. The ledger and allocator then split what is left across columns, marginals and future releases, and hand that split to the generator bank, where three engines share one accountant and stay interchangeable. Everything after generation sits inside an assurance layer that runs on every release rather than on request. The auditor recovers a measured lower bound from the planted canaries. Four attacks run against the output and against a control group we held back. A utility evaluator scores accuracy, marginal fidelity, correlation structure and fairness drift. The frontier engine turns all of it into a curve and a signed certificate, and the measured utility feeds back so the next allocation is better informed. The console puts the curve, the ledger and the dossier side by side.

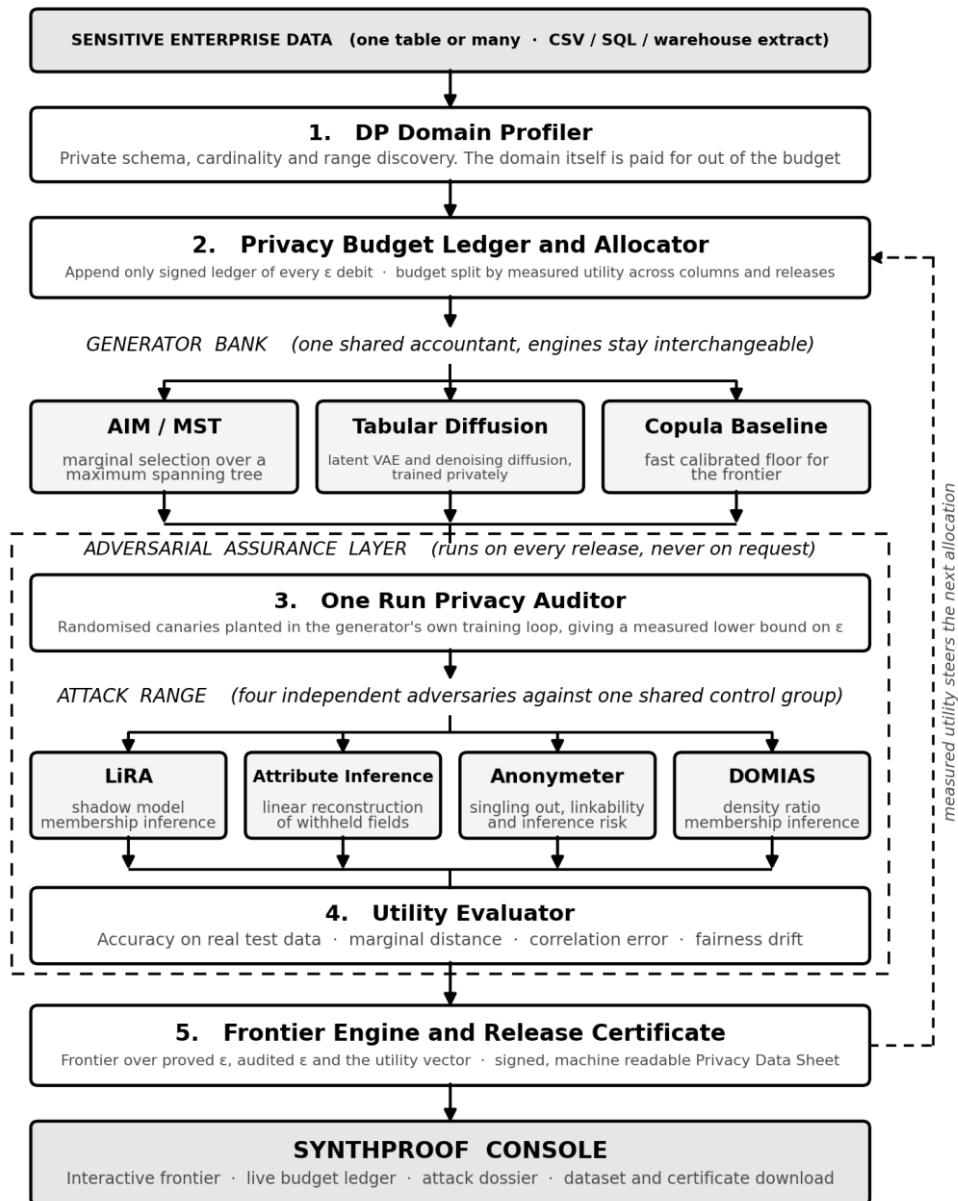


Fig 1: SynthProof. Private synthesis with an adversarial assurance layer wired into every release

Future Scope

- Multi table and relational synthesis under one global budget, keeping foreign keys and cross table correlations intact. That is where most real enterprise schemas actually live, and it is where the current open tooling is weakest.
- Taking the auditing layer beyond tabular generators to text and time series, including private fine tuning of language models on sensitive corpora, where canary based auditing already has a well studied analogue.
- A federated mode where several institutions contribute to one synthetic release without any of them seeing another's raw records, combining secure aggregation with the accountant we already have.
- Certifying the audit itself. Moving from a lower bound produced by one specific attack to a distribution free confidence interval on what any attacker could achieve, which would let the platform state a genuine two sided guarantee instead of a bound and an observation.
- An export mode aimed at regulators, rendering the ledger and the release certificates into the evidence formats used in data protection impact assessments, so the artefact a compliance team needs comes out of the pipeline instead of being rebuilt afterwards.